Docker

1. Docker & Containers vs. VMs

Before getting into how Docker and Containers differ from VMs, it helps to understand virtualization first, since that's the core idea behind VMs.

Why not just install everything directly on hardware?

Imagine you have an app that needs a specific OS, plus certain dependencies and packages. Normally, that means buying a physical machine and installing everything on it directly.

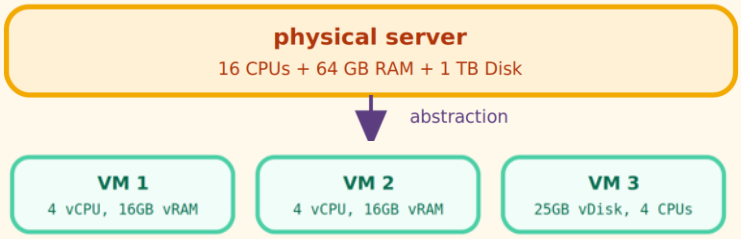
Now say you want to deploy that same app somewhere else. Will the exact same setup work on a completely different machine? Not necessarily — the new machine might have a different OS, and matching everything from the first machine exactly, down to the same dependency versions, is genuinely difficult. This is the dependency problem: apps become fragile and tightly tied to one specific environment.

On top of that, there's a design decision hiding here too: is the machine meant to run just one app, or several? Different apps often need different dependency versions, which can conflict with each other on the same machine.

And then there's horizontal scaling. If a server can't handle the load alone, you scale out by adding more machines — but each new machine needs its own full OS installed and configured, wasting time and resources. Buying a dedicated physical server for every single app is also extremely wasteful: most servers sit mostly idle, yet still consume power, need maintenance, and tie up resources that could be shared.

Enter virtualization

Say we have a powerful physical server with 16 CPUs, 64 GB of RAM, and a 1 TB disk. Virtualization means splitting up that one piece of hardware and presenting it as several separate virtual machines — for example, one VM getting 25 GB of disk, 16 GB of RAM, and 4 CPUs, and so on for the others.



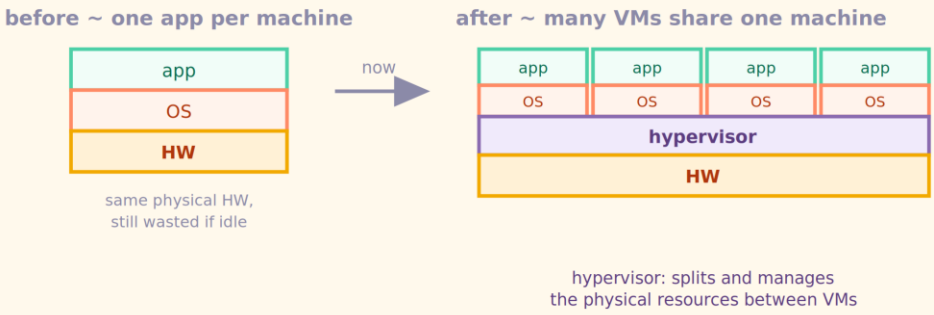
~ this abstraction layer is what lets a single physical machine be treated as several independent 'machines'.

---

2. What is a hypervisor?

Traditionally, a single machine ran one OS directly on the hardware, with one app on top of that OS.

But we can now have several VMs running side by side, each thinking it has its own dedicated hardware, even though they're all quietly sharing the same physical machine underneath.



Each VM behaves as if it's an entirely separate, self-contained server — with its own OS, its own slice of resources, and no awareness that it's actually sharing hardware with anyone else.

So who decides how those resources get split between the VMs, and who manages the VMs themselves? That's exactly the hypervisor's job: it sits between the hardware and the VMs, splitting up the physical resources and handing each VM its own virtual share.

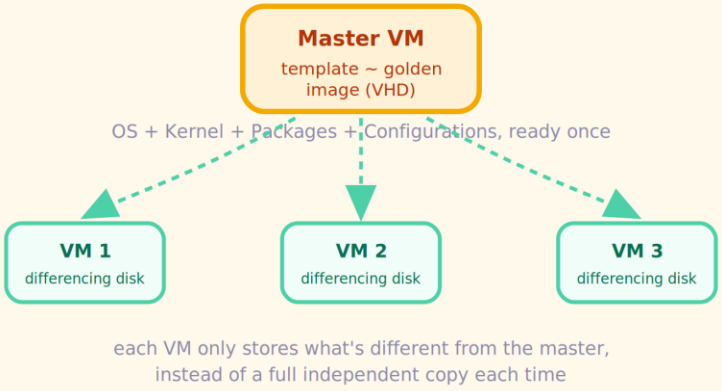
Practically, if you make a VM from a Linux ISO, its kernel is virtual too, and the same goes for its CPU, RAM, and disk — they all become vCPU, vRAM, and vDisk. This is exactly what gives VMs isolation from each other.

~ VMs bring real advantages: strong isolation, and resource scaling that feels closer to unlimited compared to a single physical server. But they also come with real downsides: setting one up from scratch takes more effort and configuration , even if it isn't as bad as building a full physical server from zero.

---

3. Reducing the setup overhead: VM templates

Regarding the setup overhead — if you already have a VM with the OS, kernel, and all the needed packages and configurations installed, you can turn it into a Template known as a Master VM (or Golden Image / VHD): a ready-made starting point.



Any new VM can then be created from that master using a differencing disk, meaning it only stores what's different from the master image, instead of duplicating a full, independent copy every single time.

A quick comparison with traditional servers

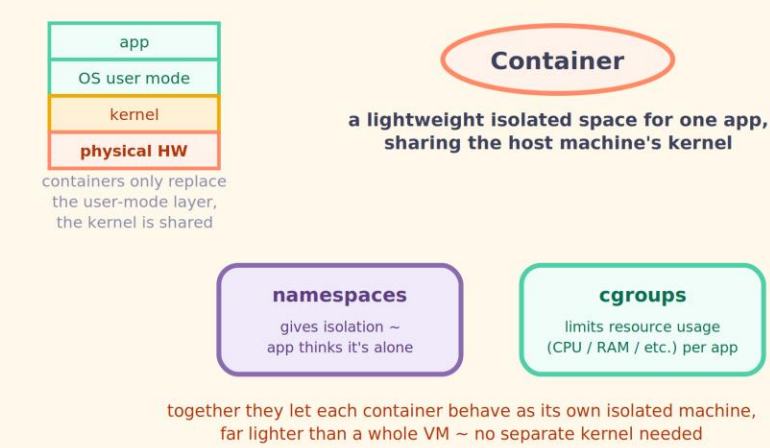
With Traditional servers, each one is fully independent, so restoring or migrating between servers is not straightforward, since it means dealing with mismatched setups, networking, and backups separately.

Since VMs are essentially just files, moving or restoring them is much simpler by comparison, though there's still some overhead: VM licensing, and the fact that each VM carries its own full OS, which means some duplicated resource usage across VMs on the same host.

---

4. So what exactly is a Container?

Compared to a VM's stack (App / OS / Hypervisor / HW), what does a container actually replace?



A container is a lightweight, isolated space for one app, and unlike a VM, it doesn't carry its own kernel — it shares the host machine's kernel directly. This is exactly why containers are so much lighter and faster to start than VMs.

This isolation and resource control is made possible mainly through two Linux kernel features:

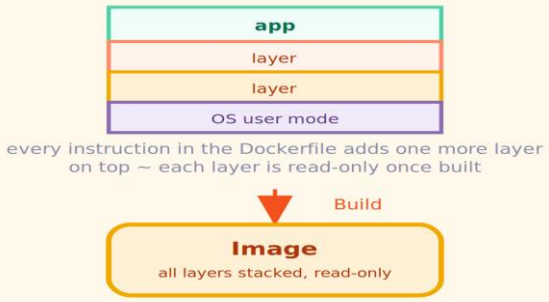
- Namespaces — give the container its isolation: from inside, the app genuinely believes it's running alone on its own machine.
- Cgroups (control groups) — limit how much of the host's resources (CPU, RAM, etc.) each container is allowed to use, so one container can't starve the others.

~ this combination is what makes a container behave like its own little isolated machine, without the overhead of a full separate OS underneath it.

---

5. Images and layers

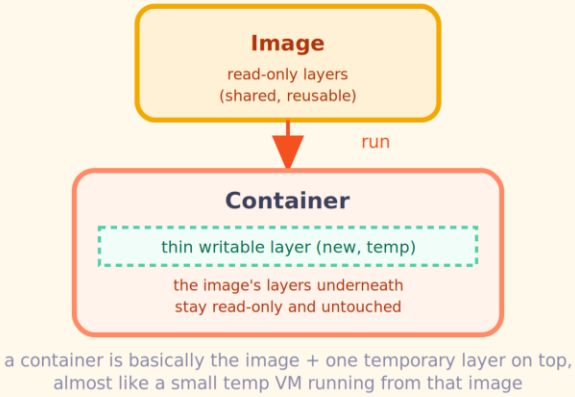
So how is a container actually built? Every layer added on top depends on the layer right before it, and if even one layer changes, every layer built on top of it has to be rebuilt too. That's exactly why we want each layer to be meaningful and correctly ordered — so that only what actually needs to change gets rebuilt, and everything unaffected can be reused as-is.



Each instruction in a Dockerfile effectively becomes one more layer stacked on top of the previous ones. Once you run a Build, all of those layers get bundled together into an Image. An image is essentially a temporary VM made up of these stacked, read-only layers.

From image to running container

To actually run one of these containers, we need to take that image, copy it, and turn it into a running instance — this running instance is what we call a Container.

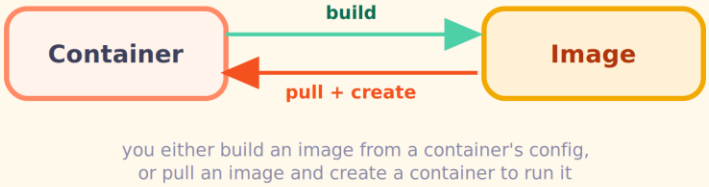


More precisely: we take the image, copy the VM's data from it, add one new configuration/writable layer on top, and start the container from there. Everything below that new layer is read-only and shared straight from the image; only that thin top layer is temporary and writable.

Once a container is running, it's essentially: read-only layers from the image, plus one temporary read-write layer sitting on top, where anything the running app writes or changes actually gets saved.

Getting from an image to a container

There are really two directions here:



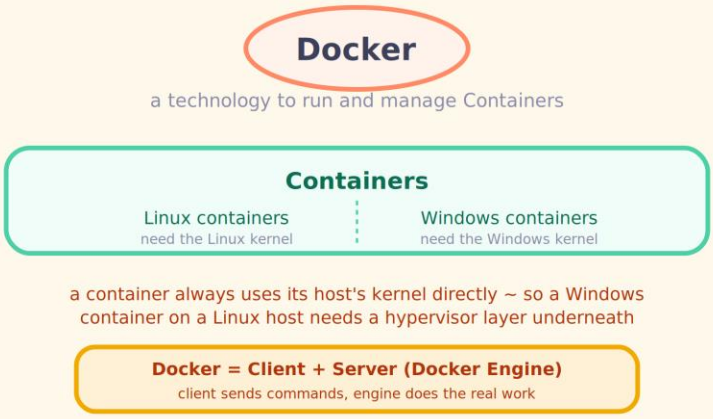
- Build — you build an image starting from a Dockerfile / container configuration.
- Pull + Create — you pull an existing image, then create a container from it to run it.

~ Pulling an image is really about getting everything ready to create a container from it, and creating/starting that container is what actually brings the app to life.

---

6. What is Docker?

Docker is a technology that helps you manage Containers easily.



Containers actually come in two flavors: Linux containers and Windows containers. Linux containers need the Linux kernel, and Windows containers need the Windows kernel.

> *~ a container always runs directly on its host's kernel. So a Windows container on a Linux host still needs a hypervisor layer (a VM) underneath to provide that Windows kernel.*

Docker itself isn't one single program. It has a Docker Engine, which is what actually does the work (pulling images, running containers, etc.) so we don't have to handle all of that manually. Docker is really two pieces:

- Client — what we use to type commands.
- Server (the Engine) — the part that receives those commands from the client and executes them.